Robotics & Control Systems Assessment

Name: Adithya B
Role: Robotics Engineer Candidate
Tools Used: ROS 2 Humble, Gazebo, RViz, Nav2

1. Simulation Overview

In this project, the complete robot system is developed and tested in a fully simulated environment using ROS 2, Gazebo, and RViz2. Instead of relying on physical hardware, the robot operates inside a virtual world where all sensors, movements, and interactions are realistically modeled. This approach allows safe, repeatable, and efficient testing of autonomous navigation algorithms.The primary goal of the simulation is to enable the robot to navigate from a starting position to a user-defined goal while avoiding obstacles, using LiDAR data as its main sensing input. The simulation mimics a real indoor environment, where the robot must continuously perceive its surroundings, make decisions, and adjust its motion accordingly.

1.1 Simulation Workflow

The overall workflow of the system can be described as follows:

- The robot is spawned inside a Gazebo environment.
- The LiDAR sensor continuously scans the surroundings and publishes data.
- SLAM Toolbox processes this data to create a map of the environment.
- Nav2 uses the generated map to plan a path to the target location.
- The robot follows this path while continuously monitoring for obstacles.
- If an obstacle is detected, the system reacts and adjusts the robot’s motion in real time.

1.2 Visualization of Simulation

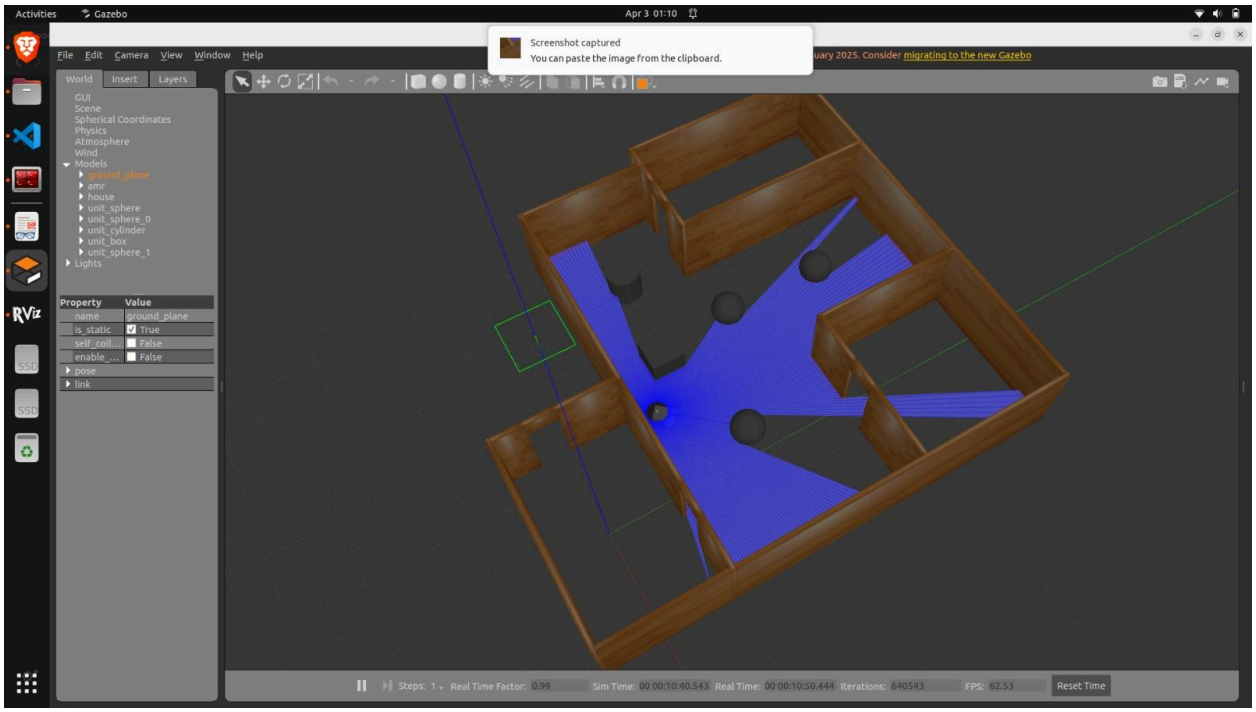


Fig.1 Gazebo simulation of a house and the robot with the lidar scan

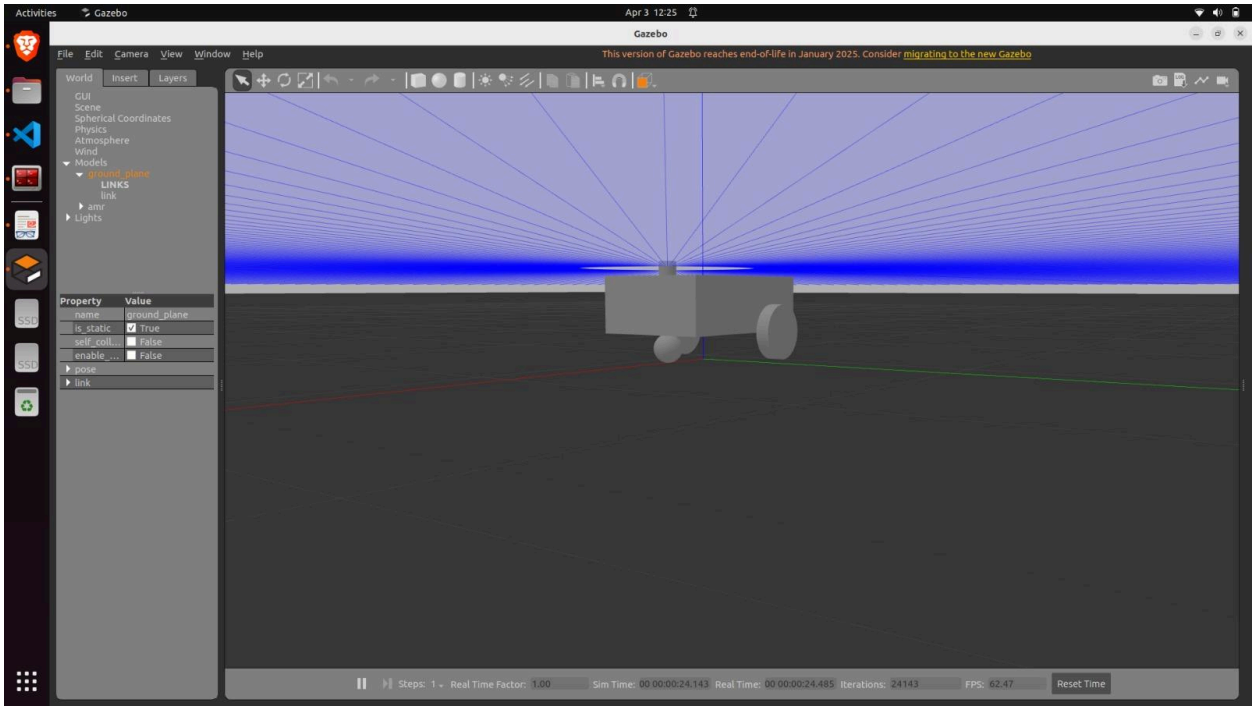


Fig.2 Robot Description

2. Simulation Setup

This section explains how the simulation environment was built and configured to enable autonomous navigation and obstacle avoidance. The setup involves creating a robot model, integrating it into Gazebo, configuring sensors, and visualizing everything in RViz.The goal of this setup is to replicate a realistic robotic system where sensing, planning, and control work together seamlessly.

2.1 Robot Model Design

The robot model is defined using a URDF (Unified Robot Description Format) file. This file describes:

- The physical structure of the robot
- The position of each component
- The relationship between different coordinate frames (TF frames)
- Differential Drive Plugin
    - This plugin converts velocity commands (`/cmd_vel`) into wheel rotations, allowing the robot to move forward, backward, and turn using the teleop_twist_keyboard

- LiDAR Plugin (Ray Sensor)
    - his plugin simulates the LiDAR sensor by generating distance measurements of surrounding objects and publishing them to the `/scan` topic used for mapping the environment and for real time obstacle avoidance

2.2 Visualization in RViz2

RViz2 is used to visualize and monitor the robot's behavior in real time. It plays a critical role in debugging and understanding how the system is working internally.

- **Robot Model** → Shows position and orientation
- **Laser Scan** → Displays LiDAR data points
- **Map** → Generated using SLAM Toolbox
- **Costmaps** → Represent obstacles and free space
- **Planned Path** → Path generated by Nav2

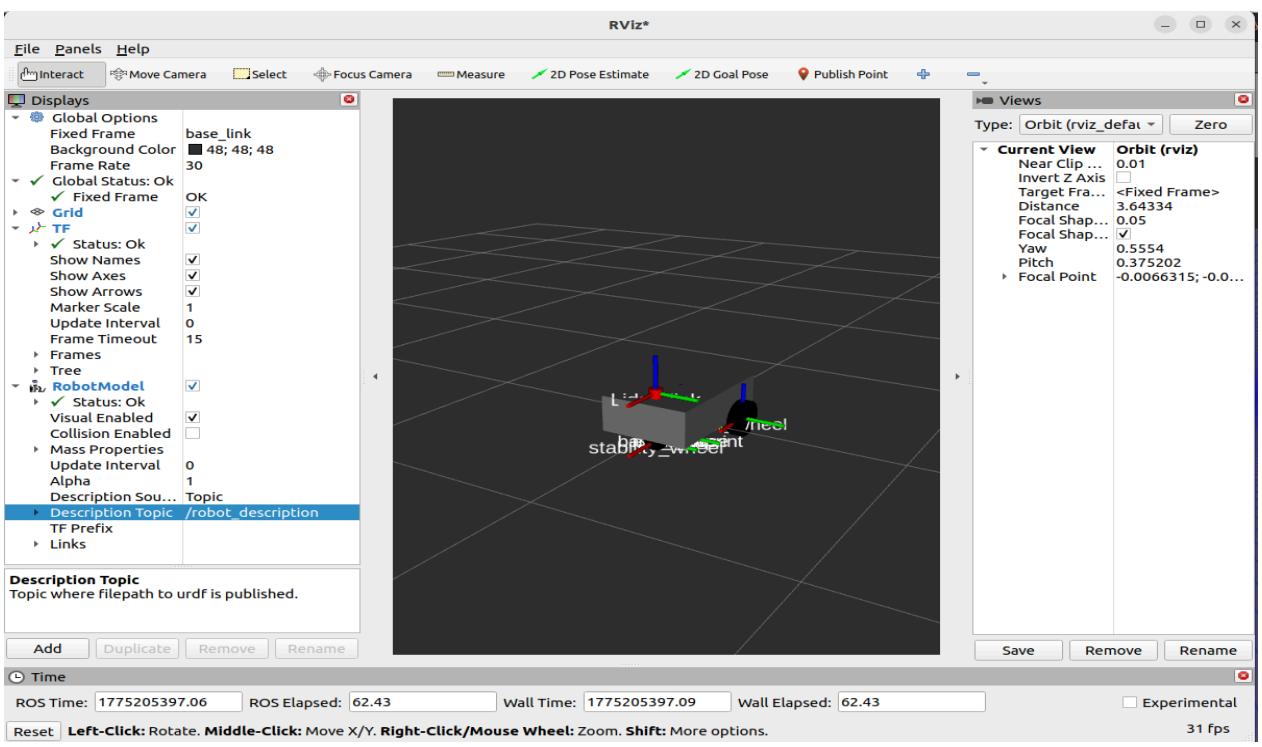


Fig.3 Robot model in rviz2

3. Mapping & Localization (SLAM Toolbox and Nav2)

SLAM Toolbox is used to allow the robot to understand and navigate its environment without any pre-existing map. SLAM (Simultaneous Localization and Mapping) enables the robot to build a map while also estimating its own position within that map

3.1Mapping.

To create the map:

- The robot is manually controlled using `teleop_twist_keyboard`
- As it moves, the LiDAR continuously scans the surroundings
- SLAM Toolbox processes this scan data and gradually builds a 2D occupancy grid map

This map represents:

- Free space (where the robot can move)
- Obstacles like walls and object with thick black lines

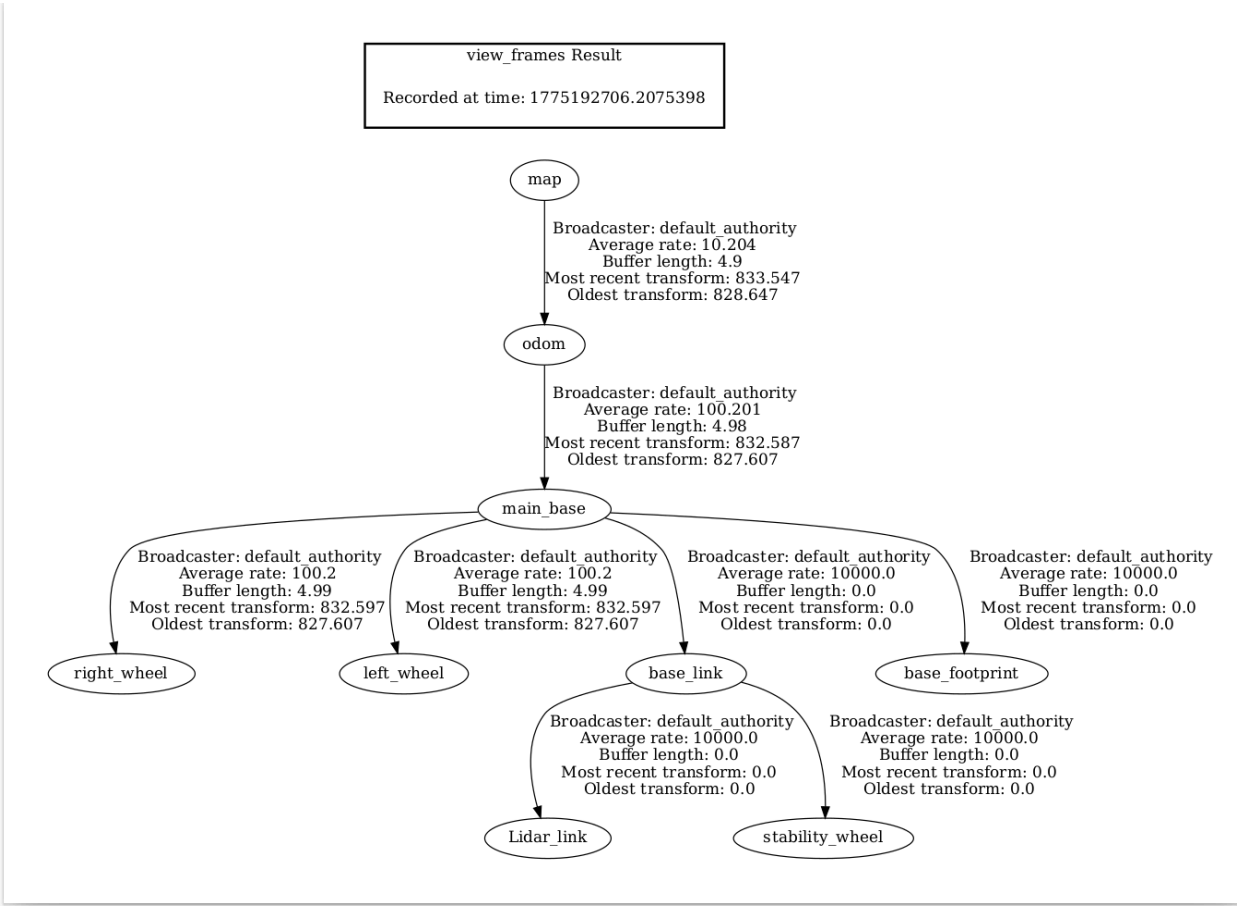
Fig.4 Robot after mapping the environment

3.2 Localization

After the map is created, the robot uses it to localize itself, meaning it continuously estimates its position inside the map.

- The robot compares real-time LiDAR scans with the saved map
- SLAM Toolbox updates the robot's pose (position and orientation)
- This allows accurate tracking even while the robot is moving

4. TF Frame Structure



The TF (transform) structure defines how all parts of the robot and the environment are connected through coordinate frames, ensuring consistent spatial understanding during navigation. In this system, the hierarchy follows **map → odom → main_base → base_link → lidar_link**, where the *map* frame represents the global environment created by SLAM Toolbox, *odom* tracks the robot's motion over time, and *base_link* represents the robot's physical body used for control and navigation. The *lidar_link* provides sensor data for obstacle detection, which is transformed through this chain to align with the robot and map. Additional frames like wheels and base footprint support movement and stability. This structured TF tree ensures accurate localization, correct sensor alignment, and smooth operation of Nav2 and obstacle avoidance.

5.State Machine Design (CORE PART)

The following states are used in the system:

- IDLE
  The initial state where the robot is waiting to start.
- INITIALIZE
  Activates required nodes such as Nav2 and SLAM Toolbox.
- PLAN_PATH
  A navigation goal is sent, and the system computes a path.
- MOVE_TO_GOAL
  The robot follows the planned path using Nav2.
- OBSTACLE_DETECTED
  Triggered when LiDAR detects an obstacle within a threshold distance.
- AVOID_OBSTACLE
  The robot temporarily overrides navigation and performs a safe maneuver.
- GOAL_REACHED
  The robot stops after reaching the target.

5.1 State Transition Logic

- The system starts in IDLE, then moves to INITIALIZE when execution begins.
- Once all components are active, it transitions to PLAN_PATH to compute a route.
- After planning, it enters MOVE_TO_GOAL and starts moving.
- While moving, if LiDAR detects an obstacle, the system switches to OBSTACLE_DETECTED.
- It then enters AVOID_OBSTACLE, where corrective motion is applied (e.g., rotation).
- Once the path is clear, the robot returns to MOVE_TO_GOAL and continues.
- Finally, when the goal is reached, it transitions to GOAL_REACHED and stops.

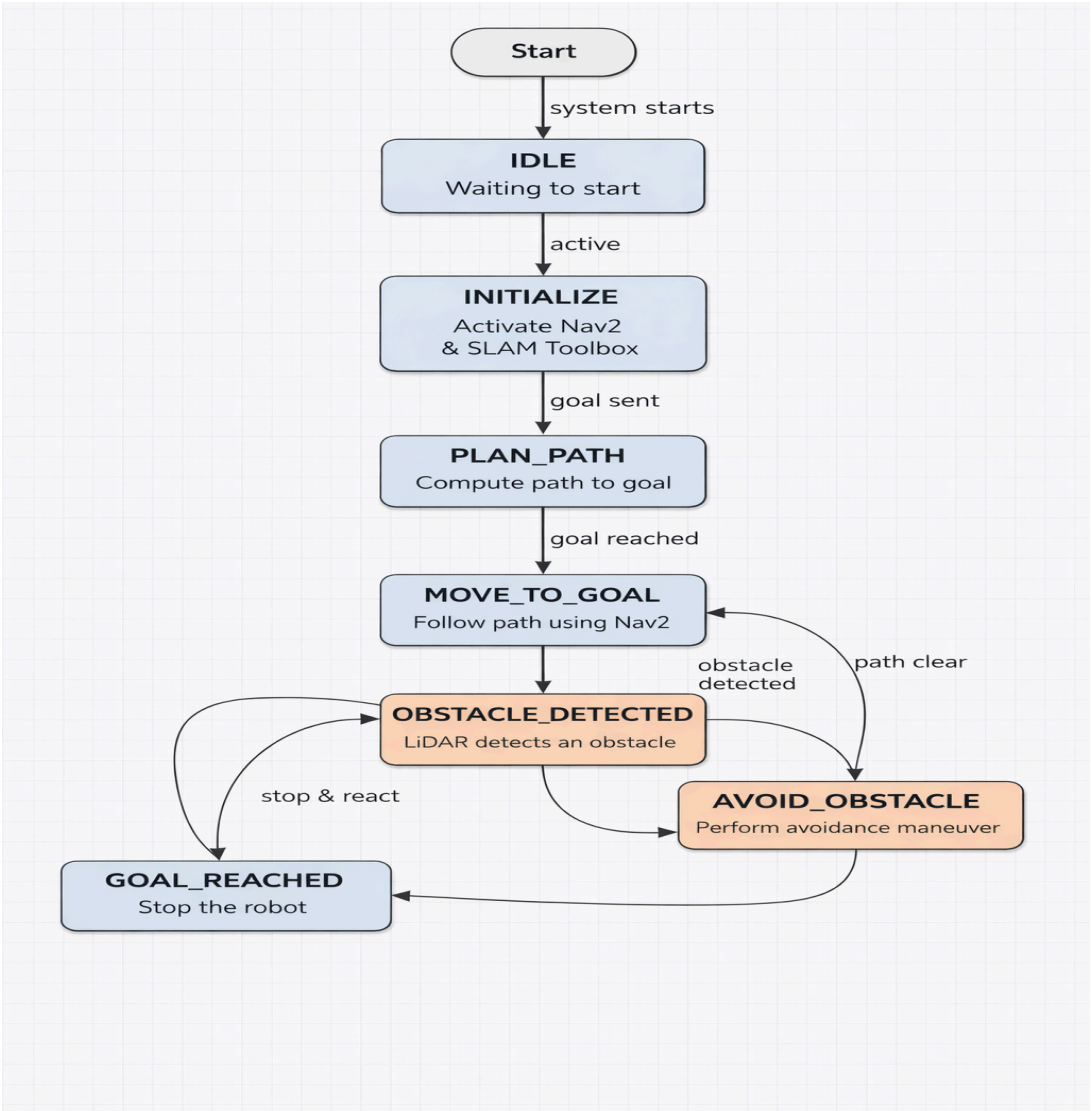


Fig 5. State Machine Visualization

Summary

The implementation of the state machine ensures that the robot Operates in a structured and predictable manner ,Responds effectively to obstacles and Maintains smooth and safe navigation

6. Simulation Script (ROS2 Logic)

The core functionality of the robot is implemented using a ROS 2-based control script, which acts as the brain of the system. This script is responsible for handling sensor data, making decisions using the state machine, and controlling the robot's movement in real time.Instead of directly controlling the robot manually, the system runs as an autonomous ROS2 node that continuously processes inputs and publishes commands.

6.1 How the Script Works

The control script runs continuously in a loop, where the robot keeps observing its surroundings using LiDAR data, checks its current state in the state machine, and then decides what action to take whether to keep moving toward the goal, stop, or avoid an obstacle. Based on this decision, it sends the appropriate velocity commands to control the robot's motion. Because this process repeats rapidly, the robot is able to respond quickly and adapt in real time to any changes or obstacles in its environment.

6.2 Integration with State Machine

The script is tightly integrated with the state machine:

- In MOVE_TO_GOAL - Nav2 controls movement
- In OBSTACLE_DETECTED - script interrupts motion
- In AVOID_OBSTACLE - script sends custom velocity commands

Pseudocode

```
if state == "MOVE_TO_GOAL":

    if min(scan_ranges) < 0.5:

        state = "OBSTACLE_DETECTED"

elif state == "OBSTACLE_DETECTED":
```

```
    stop_robot()
    state = "AVOID_OBSTACLE"
elif state == "AVOID_OBSTACLE":
    rotate_robot()
    if path_clear():
        state = "MOVE_TO_GOAL"
```

Summary

The ROS2 simulation script acts as a decision-making layer that connects perception and control. By combining LiDAR input, state machine logic, and velocity commands, the robot is able to navigate autonomously and safely in a dynamic environment

7. Results

The simulation was successfully executed in Gazebo and visualized in RViz2, demonstrating reliable autonomous navigation and obstacle avoidance.The robot was able to create a map of the indoor environment using SLAM Toolbox by teleoperating it with the keyboard. The generated map accurately represented walls and obstacles in the simulated house environment. After saving the map, the robot used Nav2 to navigate autonomously to a given goal position.During navigation, the robot continuously monitored LiDAR data and was able to detect obstacles in real time. When an obstacle was encountered, the state machine triggered the avoidance behavior, allowing the robot to safely adjust its path and continue toward the goal without collision.The TF tree remained stable throughout the simulation, ensuring proper coordination between mapping, localization, and control. The robot maintained accurate localization and smooth movement while following the planned path.

GITLINK

https://github.com/AdithyaB1509/Autonomous-Navigation-of-an-AMR.git